Proyecto No. 1: Simulación de Planificador

Cruz Morales Pablo

Sistemas Operativos en Tiempo Real
Universidad Nacional Autonoma de Mexico
Facultad de Ingenieria

Resumen:
En este proyecto, se elabora una simulación de un planificador de procesos Shortest Job First (SJF). La simulación se centrará en modelar el comportamiento del planificador SJF en un entorno controlado, permitiendo observar cómo se manejan diferentes cargas de trabajo .

Índice

1. Objetivo 1
2. Introducción 1
3. Planteamiento del Problema 1
4. Desarollo 2
5. Resultados Experimentales 3

1. OBJETIVO

Comprender el funcionamiento de los sistemas operativos a través de la simulación de un sistema operativo específico, lo que permitirá explorar su gestión de recursos y operación interna.

2. INTRODUCCIÓN

En el presente proyecto vamos a abordar, los tipos de planificadores , haciendo énfasis en el planificador SFJ, para esto debemos retomar el concepto de planificador, En pocas palabras "El planificador (scheduler) es la función fundamental de un SO, siendo el CPU uno de los principales recursos, el planificador debe decidir a qué proceso le cede los recursos del CPU cuando están listos." (Escobar, 2024).

Hay varios tipos de algoritmo de planificador dentro de los cuales los más importantes se encuentran

- Round Robin
- First Coming First Served
- Short Job First
- Priority

Como previamente se menciono el proyecto se enfocaremos exclusivamente en el Short Job First, en este tipo de algoritmo de planificador se le da prioridad de ejecución al procesoque tenga una duración menor,esto se puede ver explícitamente en la imagen 1.

La figura uno muestra cuatro procesos, en este el proceso dos al ser el de menor duración es el primero en ser ejecutado, terminado dicho procesos se le da el paso al tercer proceso para ser ejecutado y así sucesivamente hasta que se le dé paso al proceso de mayor duracion (el primer proceso) .

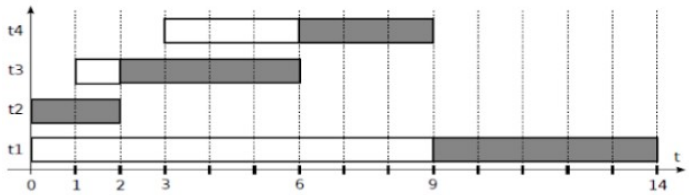


Figura 1. Imagen 1

3. PLANTEAMIENTO DEL PROBLEMA

Para hacer una re-creación de un planificador de algoritmo SFJ empezamos definiendo cómo este se comporta. Una manera simple de ver el comportamiento se observa en la imagen 2.

Este muestra el comportamiento que debemos recreardel planificador en forma de diagrama de flujo.

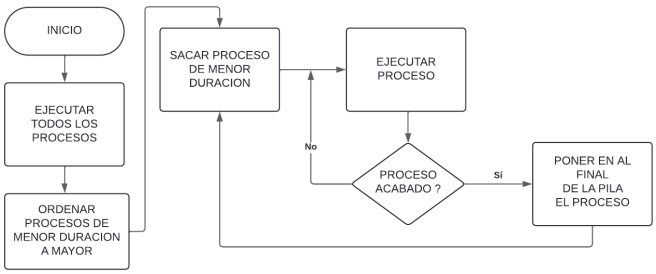


Figura 2. Imagen 2

Pero por fines academicos en el proyecto se les hará un par de modificaciones. Estas modificaciones es el añadir quantums, el quantum es "el tiempo mínimo que se permitirá a un proceso el uso del CPU" (Escobar, 2024).

Al momento de añadir el manejo de tiempo ,Debmos considerar la situación en la que el tiempo de ejecución de los quantums es menor que el tiempo total de duración del proceso. En este caso, si el proceso que está en ejecución no ha finalizado, es necesario guardar su contexto. El contexto se refiere a los valores contenidos en los registros del procesador y en la memoria de trabajo del código que se está ejecutando.

Para almacenar esta información, se debe utilizar una pila (stack) que contenga la memoria de trabajo de todos

los procesos. Y ademnas de considerar otra situacion donde el proceso que no acabo su ejecucio, al ser nuevamente ejecutada pueda recuperar el contexto guardado en el stack.

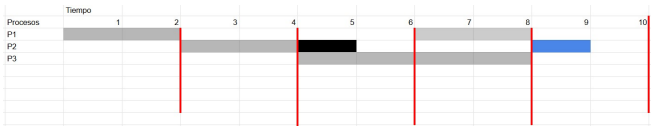


Figura 3. Imagen 3

En la imagen Tres podemos observar las situaciones exphuestas anteriormente , en esta imagen se muestran tres procesos p1,p2,p3.

Cada proceso tiene diferete duracion , como el menor es p1 será el primero en ejecutarse, Este proceso acaba antes del tiempo maximo que le dimos a nuestro planificador para cada tarea ,(represenatdo por una linea roja en la imagen), asi que lo omitimos , pero al contrario para p2 y p3 su tiempo es mayor.

Al enfocarnos solo en p3 podemos ver un rectangulo negro despues de la liena roja , este es el fragmento de progrma que falta por ejecutarse, así que entes de que cambie de proceso , guardamos el contexto de p2 y se recuperá en su segunda vuelta , representado por el rectangulo Azul.

Estas modificaciones también afectarán nuestro diagrama de flujo. El diagrama, ahora con los cambios resultantes de la integración de los quantums, se puede observar en la imagen 4.

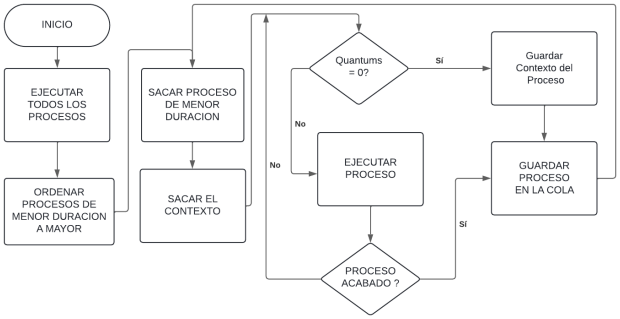


Figura 4. Imagen 4

4. DESAROLLO

Con base en lo anterior, comenzamos con el desarrollo del programa y sus consideraciones. Para este proyecto, se nos solicitó crear dos vectores de longitud n igual a mil, los cuales se utilizarán para realizar operaciones mutuas en diez distintas funciones.

Por lo tanto, el primer paso lógico fue definir las operaciones, las cuales se convertirán en nuestros procesos, dichos procesos se en listan en la siguiente lista.

- Energia del Vector A
- Energia del Vector B
- Producto Punto
- Distancia Euclidiana
- Resta de Vectores
- Producto de Vectores
- Division de Vectores
- Suma de Vectores
- Convolucion
- Convolucion con ventana triangular

Como se muestra en el Diagra de Flujo, elaboramos un programa que nos permita identificar cuál de los diez procesos es el más corto. Para lograr esto, utilizamos unas funciones que se encuentran presentes dentro de las libreias de c (start y end Clock).

```
for (int i = 0; i < 10; i++) {
    start = clock();

    // Ejecutar la operaci n (sin el tercer argumento)
    procesos[i](A, B);

    end = clock();
    tiempos[i] = ((double)(end - start)) / CLOCKS_PER_SEC;
}
```

La sección anterior de código registra el tiempo en que se inicio el proceso. Cuando el programa finaliza, este reloj se detiene y el tiempo se guarda en una variable. Posteriormente, calculándolo mediante la resta entre el tiempo de inicio y el tiempo final se imprime el tiempo que duro el proceso en ejecucion.

También contamos con una función que se encarga únicamente de asignar valores de quantums a los procesos. Esto con el fin de tener un entorno controlado para la simulacion.

Esta asignación se realiza con las siguientes líneas de código:

```
void asignar_quantums(int quantum) {
    datos.suma_vectores.quantum = quantum;
    datos.resta_vectores.quantum = quantum;
    datos.producto_vectores.quantum = quantum;
    datos.division_vectores.quantum = quantum;
    datos.distancia_euclidiana.quantum = quantum;
    datos.producto_punto.quantum = quantum;
    datos.convolucion_cuadrada.quantum = 10000;
    datos.convolucion_triangular.quantum = 10000;
    datos.energia_vector_A.quantum = quantum;
    datos.energia_vector_B.quantum = quantum;
}
```

El siguiente paso fue la creación de una estructura donde podamos almacenar los procesos que lleguen ya ordenados. Para esto, utilizamos las siguientes funciones y estructuras:

- Estructura de la pila

```c
void asignar_quantums(int quantum) {
   typedef struct {
    Operacion funciones[10];  // Almacena hasta 10 funciones
    int inicio;               //  ndice  de inicio de la FIFO
    int final;                //  ndice  de final de la FIFO
    int tamano;               // N mero de elementos en la FIFO
} Pila_Funciones;

}
```

- Creación y llenado de la Pila

```c
    // Inicializar la pila FIFO
    Pila_Funciones pila;
    inicializar_pila(&pila);


    // Agregar los procesos a la pila
    for (int i = 0; i < 10; i++) {
        agregar_funcion(&pila, procesos[i]);
    }
}
```

Ademas vamos a requerir otra estructura donde podamos almacenar los contextos, de cada proceso interrumpido.

```c
typedef struct {
    ProcesoEstado suma_vectores;
    ProcesoEstado resta_vectores;
    ProcesoEstado producto_vectores;
    ProcesoEstado division_vectores;
    ProcesoEstado distancia_euclidiana;
    ProcesoEstado producto_punto;
    ProcesoEstado convolucion_cuadrada;
    ProcesoEstado convolucion_triangular;
    ProcesoEstado energia_vector_A;
    ProcesoEstado energia_vector_B;
} DatosProcesos;
```

Siguiendo lo descrito en nuestro diagrama de flujo, entramos en un ciclo infinito donde sacamos de la pila el proceso de menor duración. Mientras los Quantums no sean cero, el programa seguirá ejecutándose. Si los quantums llegan a cero, el programa cuenta con una función que guarda el estado del proceso y lo coloca de nuevo en la pila, junto con su contexto.

Esto, en C, se implementa de la siguiente manera:

```c
printf("*********************************
    PLANIFICADOR
    ********************************************
    ");
    // Ejecutar las funciones en la FIFO hasta que est  vac a
    while (!esta_vacia(&pila)) {
        Operacion f = sacar_funcion(&pila);
        if (f != NULL) {

            resultado = f(A, B);  // Ejecutar la funci n con los vectores A,
            if (resultado == 0) {
                agregar_funcion(&pila, f);  // Si no ha terminado, volver a agregar a la pila
            }
        }
    }
```

5. RESULTADOS EXPERIMENTALES

Durante la ejecución, el programa funciona de la siguiente manera:

- 1 Creacion de los vectores y asignacion de los Quatunms

```
Vector A:
A[0] = 92.919708
A[1] = 49.809258
A[2] = 48.954739
A[3] = 35.773796
A[4] = 91.448715
A[5] = 19.333475
A[6] = 71.025726
A[7] = 99.694809
A[8] = 43.082981
A[9] = 32.880642

Vector B:
B[0] = 18.051699
B[1] = 47.648548
B[2] = 45.875423
B[3] = 43.226418
B[4] = 65.828423
B[5] = 31.748405
B[6] = 49.766533
B[7] = 47.544785
B[8] = 51.216160
B[9] = 41.712700

--- Suma de vectores ---
Indice actual: 0
Quantum: 10000

--- Resta de vectores ---
Indice actual: 0
Quantum: 10000
```

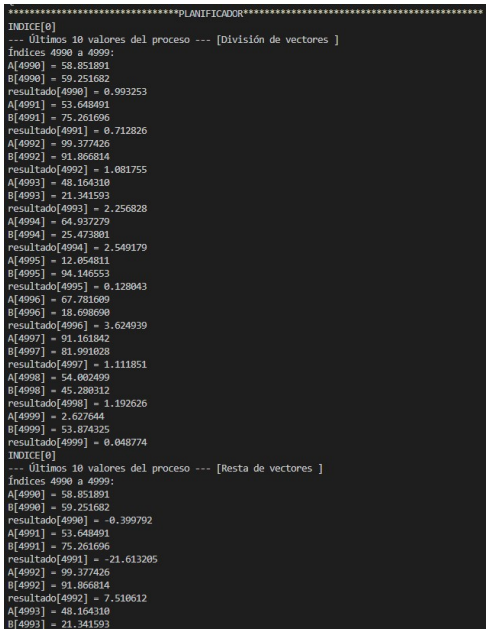
Figura 5. Imagen 5

2 Primera Corrida y Ordenamiento de los Procesos

```
--- Producto punto acumulado ---
Acumulado: 0.000000
Indice actual: 0
Quantum: 10000
INDICE[0]Tiempo de proceso 1: 0.001000 segundos
Tiempo de proceso 2: 0.000000 segundos
INDICE[0]Tiempo de proceso 3: 0.000000 segundos
INDICE[0]Tiempo de proceso 4: 0.002000 segundos
Tiempo de proceso 5: 0.000000 segundos
INDICE[0]Tiempo de proceso 6: 0.003000 segundos
INDICE[0]Tiempo de proceso 7: 0.000000 segundos
Tiempo de proceso 8: 0.000000 segundos
INDICE[0]Tiempo de proceso 9: 0.000000 segundos
Tiempo de proceso 10: 0.000000 segundos
Tiempo de proceso 1 (ordenado): 0.000000 segundos
Tiempo de proceso 2 (ordenado): 0.000000 segundos
Tiempo de proceso 3 (ordenado): 0.000000 segundos
Tiempo de proceso 4 (ordenado): 0.000000 segundos
Tiempo de proceso 5 (ordenado): 0.000000 segundos
Tiempo de proceso 6 (ordenado): 0.000000 segundos
Tiempo de proceso 7 (ordenado): 0.000000 segundos
Tiempo de proceso 8 (ordenado): 0.001000 segundos
Tiempo de proceso 9 (ordenado): 0.002000 segundos
Tiempo de proceso 10 (ordenado): 0.003000 segundos
```

Figura 6. Imagen 6

3 Ejecucion de la Simulación del Planificador



```
*****************************PLANIFICADOR*****************************
INDICE[0]
--- Últimos 10 valores del proceso --- [División de vectores ]
Índices 4990 a 4999:
A[4990] = 58.851891
B[4990] = 59.251682
resultado[4990] = 0.993253
A[4991] = 53.648491
B[4991] = 75.261696
resultado[4991] = 0.712826
A[4992] = 99.377426
B[4992] = 91.866814
resultado[4992] = 1.081755
A[4993] = 48.164310
B[4993] = 21.341593
resultado[4993] = 2.256828
A[4994] = 64.937279
B[4994] = 25.473881
resultado[4994] = 2.549179
A[4995] = 12.054811
B[4995] = 94.146553
resultado[4995] = 0.128043
A[4996] = 67.781609
B[4996] = 18.698690
resultado[4996] = 3.624939
A[4997] = 91.161842
B[4997] = 81.991028
resultado[4997] = 1.111851
A[4998] = 54.002499
B[4998] = 45.280312
resultado[4998] = 1.192626
A[4999] = 2.627644
B[4999] = 53.874325
resultado[4999] = 0.048774
INDICE[0]
--- Últimos 10 valores del proceso --- [Resta de vectores ]
Índices 4990 a 4999:
A[4990] = 58.851891
B[4990] = 59.251682
resultado[4990] = -0.399792
A[4991] = 53.648491
B[4991] = 75.261696
resultado[4991] = -21.613205
A[4992] = 99.377426
B[4992] = 91.866814
resultado[4992] = 7.510612
A[4993] = 48.164310
B[4993] = 21.341593
```

Figura 7. Imagen 7

A continuación, se presentan las tablas comparativas de los resultados, las cuales muestran los valores del proceso en un momento específicos y al final.

Tabla 1. Valores de Energía, Producto Punto y Distancia Euclidiana

Parámetro	Valor	Índices
Energía B	16,705,333	0 - 5000
Producto Punto	12,482,074	0 - 5000
Energía A	16,590,357	0 - 5000
Distancia Euclidiana	8,331,554	0 - 5000
Valor Final Energía B	33,657,720	5001 - 10000
Valor Final Producto Punto	25,014,198	5001 - 10000
Valor Final Energía A	32,905,750	5001 - 10000
Valor Final Distancia Euclidiana	4,066	5001 - 10000

Tabla 2. Comparativa de Resultados de Resta de Vectores

Índice 1	Resultado (Resta) 4999	Índice 2	Resultado (Resta) 9990
4990	-42.867	9990	-36.806
4991	-30.362	9991	-33.176
4992	7.859	9992	-2.737
4993	-12.943	9993	30.724
4994	-3.468	9994	88.383
4995	51.176	9995	15.633
4996	-17.697	9996	40.621
4997	-31.429	9997	10.316
4998	-85.595	9998	11.848
4999	-16.295	9999	24.445

Tabla 3. Comparativa de Resultados del Producto de Vectores

Índice 1	Resultado (Producto) 4999	Índice 2	Resultado (Producto) 9990
4990	5483.485	9990	484.599
4991	88.854	9991	2066.762
4992	42.620	9992	4270.047
4993	424.278	9993	5399.571
4994	3144.892	9994	242.887
4995	2639.399	9995	2451.581
4996	548.844	9996	5841.200
4997	4855.167	9997	414.189
4998	450.562	9998	4835.771
4999	1537.352	9999	3883.098

Tabla 4. Comparativa de Resultados de la División de Vectores

Índice 1	Resultado (División) 4999	Índice 2	Resultado (División) 9990
4990	0.565	9990	0.218
4991	0.081	9991	0.489
4992	3.130	9992	0.959
4993	0.539	9993	1.515
4994	0.940	9994	34.132
4995	2.609	9995	1.369
4996	0.478	9996	1.691
4997	0.639	9997	1.651
4998	0.055	9998	1.186
4999	0.662	9999	1.477

Tabla 5. Comparativa de Resultados de la Suma de Vectores

Índice 1	Resultado (Suma) 4999	Índice 2	Resultado (Suma) 9990
4990	154.180	9990	57.386
4991	35.739	9991	96.787
4992	15.240	9992	130.720
4993	43.182	9993	150.141
4994	112.212	9994	93.718
4995	114.789	9995	100.253
4996	50.086	9996	158.161
4997	142.858	9997	41.990
4998	95.545	9998	139.583
4999	80.093	9999	127.004

REFERENCIAS

[1] Escobar, L. E. (2024). *Sistemas operativos en tiempo real* [Diapositivas].

[2] Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). New Jersey: Prentice Hall.